

LIVRABLE 1

Architecture, modélisation et pipeline ELT



NUTRISCAN
ANALYSES & INSIGHTS NUTRITIONNELS

TABLE DES MATIÈRES

Table des matières	2
Table des Figures	3
1. Introduction	5
1.1. Machine utilisée.....	5
1.2. Besoins utilisateurs	5
2. Architecture Data Lake	7
2.1. Architecture médaillon	7
2.2. Stack technique.....	7
2.3. ELT vs ETL	8
3. Schéma dimensionnel	8
3.1. Schéma en étoile	8
3.2. DDL DuckDB	9
3.3. Vues DuckDB	10
3.3.1. V_food	10
3.3.2. V_food_flat.....	10
4. Data catalogue	11
4.1. Table de Faits : FACT_NUTRITION	11
4.2. Table de Dimension : DIM_PRODUCT	12
4.3. Table de Dimension : DIM_CATEGORY	13
4.4. Table de Dimension : DIM_COUNTRY	13
4.5. Table de Dimension : DIM_BRAND	14
Table de Dimension : DIM_DATE.....	14
5. Data Quality	16
5.1. Rapport data quality initial	16
1. Taux de complétude	16
2. Anomalies détectées.....	17
3. Requêtes de mesure	19
6. Pipeline ELT (Apache Hop)	22
6.1. Étape préalable : Préparer le fichier source pour Hop	22
6.2. Description des composants	23
6.3. Chargement des données brutes sur raw	26
6.4. Versionnement et logs d'exécution	27

7. Base de données NoSQL MongoDB.....	29
7.1. Justification du choix	29
7.2. MongoDB chargé.....	30
7.3. Structure JSON documentée pour la collection mobile	30
7.4. Requêtes MongoDB.....	32
7.5. Collection scan_logs et requête alternatives	36
8. Conclusion.....	38
9. Instrument de recherche.....	38
9.1. Fiche démographique — à remplir S1 lundi matin	38
9.2. Auto-positionnement — à remplir deux fois.....	39
9.3. Questionnaire CLT — S1 + S2	40
9.4. Grille d'auto-observation — S1 + S2.....	42
9.5. Journal de bord — S1 + S2	43

TABLE DES FIGURES

Figure 1 : Information système de la machine d'exécution.....	5
Figure 2 : Schéma d'architecture médaillon	7
Figure 3 : Schéma dimensionnel en étoile	9
Figure 4 : 0 données - DDL	10
Figure 5 : 6 tables - DDL.....	10
Figure 6 : Describe v_food.....	10
Figure 7 : Describe v_food_flat	11
Figure 8 : Pipeline ELT sur Apache Hop	22
Figure 9 : Composant Data grid - Meta	23
Figure 10 : Composant Data grid - Data	23
Figure 11 : Composant Parquet File input.....	24
Figure 12 : Composant Parquet File output	25
Figure 13 : MongoDB output – output options.....	26
Figure 14 : MongoDB output – Mongo document fields	26
Figure 15 : Résultat Chargement données vers raw	27
Figure 16 : Propriété fichier food_clean_date.parquet	28
Figure 17 : Résultat vérif si food_clean_date lisible	28

Figure 18 : Requête test sur pipeline_logs.....	29
Figure 19 : Résultat requête findone() - MongoDB.....	30
Figure 20 : Requête 1 - MongoDB	33
Figure 21 : Requête 2 - MongoDB	33
Figure 22 : Requête 3 – MongoDB	34
Figure 23 : Requête avant index – MongoDB	35
Figure 24 : Requête après index – MongoDB.....	35
Figure 25 : Requête d’agrégation – MongoDB.....	36
Figure 26 : Script collection scan_logs - MongoDB	37
Figure 27 : Résultat du script scan_logs	37

1. INTRODUCTION

Ce livrable porte sur la conception de la solution et la mise en place du pipeline d'ingestion. Il doit démontrer la compréhension de l'architecture Data Lake et la capacité à concevoir un modèle dimensionnel cohérent.

1.1. MACHINE UTILISÉE

 Processeur AMD Ryzen 7 5800U with Radeon Graphics 1.90 GHz	 Mémoire RAM installée 16,0 Go Vitesse : 4266 MT/s	 Carte graphique 2 GB AMD Radeon(TM) Graphics	 Stockage 954 GB 427 GB sur 954 GB utilisé
---	--	---	--

Figure 1 : Information système de la machine d'exécution

1.2. BESOINS UTILISATEURS

Profil 1 : Nutritionniste / Professionnel de sante

Ce profil souhaite analyser la qualité nutritionnelle des produits alimentaires pour guider des recommandations de sante. Les besoins identifiés sont les suivants :

- Comparer la qualité nutritionnelle des produits selon leur type et leur origine géographique.
- Identifier les produits et les catégories les plus et les moins favorables sur le plan nutritionnel.
- Analyser la composition détaillée des produits (teneurs en graisses, sucres, sel, protéines, fibres, énergie).
- Evaluer le niveau de qualité nutritionnelle globale d'un produit selon les systèmes de notation existante.
- Mesurer le degré de transformation industrielle des produits alimentaires.
- Observer les évolutions de la qualité nutritionnelle dans le temps.
- Comparer les profils nutritionnels entre différents pays ou régions.

Profil 2 : Analyste marche / Grande distribution

Ce profil souhaite comprendre la structure du marché alimentaire pour orienter les décisions commerciales. Les besoins identifiés sont les suivants :

- Identifier les marques dominantes par catégorie de produits et analyser leur positionnement nutritionnel.
- Comparer les performances nutritionnelles des grandes marques sur une catégorie donnée.
- Mesurer le niveau de transformation des produits par marque et par catégorie.

- Suivre l'évolution du nombre de références et de la qualité nutritionnelle sur plusieurs années.
- Identifier les tendances de marche émergentes à partir des données de création de produits.

Profil 3 : Utilisateur application mobile NutriScan

Ce profil est un consommateur qui utilise l'application mobile NutriScan pour obtenir des informations nutritionnelles instantanées sur un produit. Les besoins identifiés sont les suivants :

- Rechercher un produit instantanément par son code-barres EAN.
- Consulter le profil nutritionnel complet d'un produit : Nutri-Score, valeurs nutritionnelles clés.
- Découvrir des alternatives mieux notées dans la même catégorie.

2. ARCHITECTURE DATA LAKE

2.1. ARCHITECTURE MÉDAILLON

L'architecture médaillon organisée en 3 couches permet de structurer le stockage et le traitement des données au sein du Data Lake.

Bronze (Raw) : Cette couche sert à stocker le fichier source food.parquet sans aucune modification, afin de conserver un historique brut inchangé.

Silver : Dans cette couche débutent les étapes de transformations et de nettoyage, c'est la partie « T » dans ELT, c'est-à-dire nettoyer, transformer et standardiser toutes les données issues de la couche Bronze.

Gold : Cette couche est l'application concrète de notre modèle en étoile, les données sont optimisées pour être visualisées, analysées, de manière épuré et structuré pour être consommé directement par Power BI.



Figure 2 : Schéma d'architecture médaillon

2.2. STACK TECHNIQUE

- **DuckDB + DBeaver** : Exploration des données, modélisation dimensionnelle ainsi que l'implémentation de notre DDL. Utilisation surtout sur le fichier source
- **Apache Hop + MongoDB** : Pipeline ELT, Extraction du fichier source parquet, et Load des données brutes dans le répertoire « raw » et dans MongoDB. Utilisation dans la couche bronze.
- **dbt Core + DuckDB + PySpark** : Partie Transform de ELT, avec les sous parties de staging, marts et semantic layer, ainsi qu'un comparatif avec PySpark. Utilisation dans la couche silver.
- **DuckDB + Power BI + MongoDB** : Requêtes OLAP vers le dashboard Power BI, ainsi que MongoDB analytique. Utilisation dans la couche gold.

2.3. ELT VS ETL

Pour le projet NutriScan, nous avons privilégié une approche ELT (Extract-Load-Transform) plutôt qu'un modèle ETL classique. Notre pipeline développé sur Apache Hop se concentre ainsi uniquement sur l'Extraction (E) et le Chargement (L), copiant le fichier food.parquet directement vers notre stockage de données brutes raw. Si nous avions choisi une logique ETL en filtrant ou en nettoyant les données dès l'ingestion, nous aurions perdu l'accès aux informations brutes originales. Pour ensuite déléguer toute la phase de Transformation à dbt Core et DuckDB, pour nettoyer et structurer efficacement nos volumes de données.

3. SCHÉMA DIMENSIONNEL

3.1. SCHÉMA EN ÉTOILE

Pour concevoir le schéma dimensionnel en étoile, je me suis basé sur méthodologie de Kimball, afin de représenter directement les besoins métiers de nos 3 profils utilisateurs type. Le tout en structures de données claires, composées de faits, de dimensions et de mesures. La table de fait FACT_NUTRITION regroupe l'ensemble des métriques quantitatives de composition et de notation. Les 5 dimensions sont des attributs descriptifs qui répondent aux questions Qui ? (Dim_Brand) Quoi ? (Dim_Product, Dim_category) Où ? (Dim_Country) Quand ? (Dim_Date) Comment ? par rapport à notre fait.

Ce schéma nous permettra de simplifier la navigation et le filtrage futur pour la visualisation dans Power BI. Il a tout d'abord été de faire le choix d'éliminer toute clé artificielle unique dans la table de faits pour la remplacer par une clé primaire composite unissant les clés étrangères des dimensions, mais comme DuckDB interdit les valeurs NULL et qu'avec une première analyse on remarque que 40% des produits n'ont pas de catégorie et 37% n'ont pas de marque. On aurait donc rejeté 2,7 millions de lignes. De ce fait, on part sur une structure avec clé de substitution artificielle (nutrition_sk).

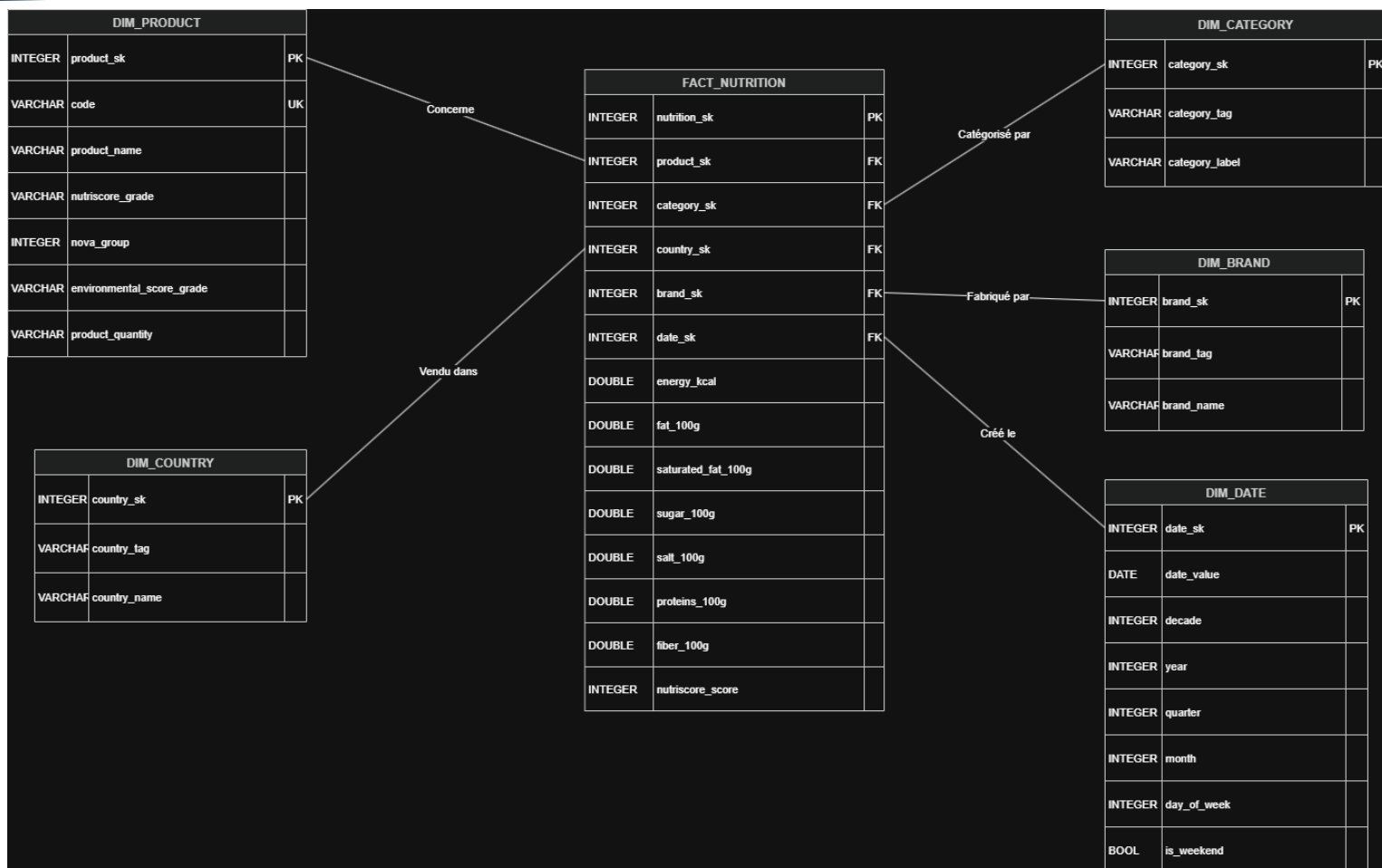


Figure 3 : Schéma dimensionnel en étoile

En explorant le fichier food.parquet, j'ai constaté que certains champs, (Ex : countries_tags), contiennent des valeurs multiples sous forme de listes pour un même produit. Il a fallu prendre ça en compte dans le choix du grain afin de ne pas perdre en performances. Je l'ai donc défini comme étant : *une ligne représente une combinaison unique entre un produit et un pays de commercialisation*.

Lors de la phase de transformation, j'utiliserais la commande « UNNEST » dans dbt Core, afin d'éclater ces listes en lignes distinctes. Il y aura donc plus de lignes en lecture, mais au lieu d'utiliser des filtres textuels lents, on utilisera des nombres entiers sur la clé country_sk. C'est En faisant cela le moteur OLAP de DuckDB pourra gérer de grandes quantités de données rapidement et donc sans perdre en performance.

3.2. DDL DUCKDB

Le DDL (Data Definition Language) est un ensemble de commandes SQL qui sert à définir, créer et modifier la structure de la base, sans manipuler.

J'ai donc créé les tables du schéma en étoile via une requête dans DuckDB. Elles sont pour l'instant vides, puis on les remplira avec dbt plus tard.

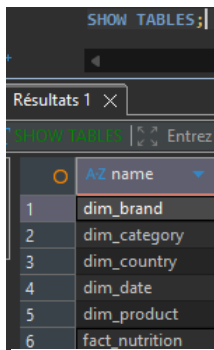


Figure 5 : 6 tables - DDL

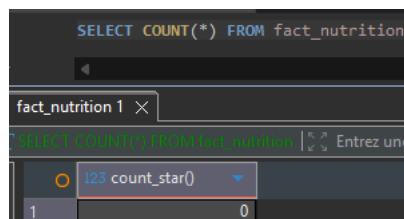


Figure 4 : 0 données - DDL

3.3. VUES DUCKDB

Il a été nécessaire de créer des vues pour simplifier les requêtes. « v_food » et « v_food_flat » ont donc été créées et sont fonctionnelles

3.3.1. V_FOOD

Au lieu d'importer les données de food.parquet dans DuckDB avec une requête qui prendrait 5 minutes et doublerait l'espace disque utilisé, du style : `CREATE TABLE AS SELECT * FROM 'food.parquet'`, À la place, on crée une vue, qui se caractérise comme étant un raccourci nommé vers une requête. L'avantage c'est qu'elle ne copie pas les données, elle pointe vers la source originale. Donc chaque fois qu'on interroge la vue, DuckDB lit directement le fichier Parquet.

	AZ column_name	AZ column_type	AZ null	AZ key	AZ default	AZ extra
1	additives_n	INTEGER	YES	[NULL]	[NULL]	[NULL]
2	additives_tags	VARCHAR	YES	[NULL]	[NULL]	[NULL]
3	allergens_tags	VARCHAR	YES	[NULL]	[NULL]	[NULL]
4	brands_tags	VARCHAR	YES	[NULL]	[NULL]	[NULL]
5	brands	VARCHAR	YES	[NULL]	[NULL]	[NULL]
6	categories	VARCHAR	YES	[NULL]	[NULL]	[NULL]
7	categories_tags	VARCHAR	YES	[NULL]	[NULL]	[NULL]
8	categories_properties	STRUCT(ciqua_food_cod	YES	[NULL]	[NULL]	[NULL]
9	food_brand_name	VARCHAR	YES	[NULL]	[NULL]	[NULL]

Figure 6 : Describe v_food

3.3.2. V_FOOD_FLAT

Les informations nutritionnelles fondamentales sont dans la colonne « nutriments » de type `STRUCT[]`, qui est un ensemble d'objets imbriqués. Ce type est illisible pour des outils comme Apache Hop et lourd à manipuler lors des requêtes, donc pour simplifier le tout, j'ai créé une vue afin d'aplatir et de simplifier la structure et en extraire chaque nutriment essentiel pour les transformer en colonnes de types simples.

Pour cela, DuckDB utilise `list_filter()` pour accéder aux tableaux imbriqués et parcourir le tableau nutriments, filtrer sur le nom, extraire la valeur 100g (au lieu de portion).

	column_name	column_type
1	code	VARCHAR
2	product_name	STRUCT(lang VARCHAR, "text" VARCHAR)
3	brands	VARCHAR
4	categories_tags	VARCHAR[]
5	countries_tags	VARCHAR[]
6	nutriscore_score	INTEGER
7	nutriscore_grade	VARCHAR
8	nova_group	INTEGER
9	environmental_score_grade	VARCHAR
10	last_modified_t	BIGINT
11	created_t	BIGINT
12	energy_kcal	FLOAT
13	fat_g	FLOAT
14	saturated_fat_g	FLOAT
15	sugar_g	FLOAT
16	salt_g	FLOAT
17	proteins_g	FLOAT
18	fiber_g	FLOAT

Figure 7 : Describe v_food_flat

4. DATA CATALOGUE

Ce data catalogue est un registre de toutes les données choisies pour notre modèle en étoile, avec leur description métier, leur structure technique, leur source et leur nom associé. Il permet à n'importe qui de comprendre rapidement ce que contient une table du schéma.

4.1. TABLE DE FAITS : FACT_NUTRITION

Centralise l'ensemble des indicateurs nutritionnels quantitatifs continus à l'échelle internationale. Une ligne correspond à une combinaison unique entre un produit et un pays de commercialisation.

Colonne	Type	Source (food.parquet)	Description métier
nutrition_sk	INTEGER	Généré automatiquement	Clé primaire unique
product_sk (FK)	INTEGER	DIM_PRODUCT	Pointe vers les attributs du produit
category_sk (FK)	INTEGER	DIM_CATEGORY	Pointe vers la catégorie de produit

country_sk (FK)	INTEGER	DIM_COUNTRY	Pointe vers les pays où le produit est commercialisé
brand_sk (FK)	INTEGER	DIM_BRAND	Clé étrangère pointant vers la marque
date_sk (FK)	INTEGER	DIM_DATE	Clé étrangère liée à la date de création du produit
nutriscore_score	INTEGER	nutriscore_score	Score nutritionnel de -15 à +40
energy_kcal_100g	DOUBLE	nutriments[name='energy-kcal']. "100g"	Énergie calorifique (en kcal) pour 100g de produit
fat_100g	DOUBLE	nutriments[name='fat']. "100g"	Quantité de graisses pour 100g
saturated_fat_100g	DOUBLE	nutriments[name='saturated-fat']. "100g"	Quantité d'acides gras saturés pour 100g
sugar_100g	DOUBLE	nutriments[name='sugars']. "100g"	Quantité de sucres pour 100g
salt_100g	DOUBLE	nutriments[name='salt']. "100g"	Quantité de sel pour 100g
proteins_100g	DOUBLE	nutriments[name='proteins']. "100g"	Quantité de protéines pour 100g
fiber_100g	DOUBLE	nutriments[name='fiber']. "100g"	Quantité de fibres alimentaires pour 100g

4.2. TABLE DE DIMENSION : DIM_PRODUCT

Cette table contient les caractéristiques descriptives du produit.

Nom de Colonne	Type	Source	Description métier
product_sk (PK)	INTEGER	Généré automatiquement	Clé primaire unique de la dimension produit
code (UK)	VARCHAR	code	Code-barres EAN du produit (pour app mobile)

product_name	VARCHAR	product_name.text	Libellé du produit alimentaire
nutriscore_grade	VARCHAR	nutriscore_grade	Lettre du Nutri-Score (A à E) pour le filtrage et les axes
nova_group	INTEGER	nova_group	Indice NOVA (1 à 4) évaluant la transformation industrielle
environmental_score_grade	VARCHAR	environmental_score_grade	Lettre du score environnemental (A à E) de l'aliment
product_quantity	VARCHAR	product_quantity, product_quantity_unit	Quantité en g ou en ml d'un produit

4.3. TABLE DE DIMENSION : DIM_CATEGORY

Cette table permet de classer et de regrouper les produits alimentaires selon les catégories Open Food Facts.

Nom de Colonne	Type	Source (food.parquet)	Description métier
category_sk (PK)	INTEGER	Généré automatiquement	Clé primaire unique de la dimension category
category_tag	VARCHAR	categories_tags	Identifiant numérique de la catégorie.
category_label	VARCHAR	categories	Intitulé de la catégorie

4.4. TABLE DE DIMENSION : DIM_COUNTRY

Cette table référence des pays où les produits sont enregistrés et distribués.

Nom de Colonne	Type	Source (food.parquet)	Description métier
country_sk (PK)	INTEGER	Généré automatiquement	Clé primaire unique de la dimension pays.

country_tag	VARCHAR	countries_tags (via UNNEST)	Tag technique d'un pays isolé après explosion de la liste.
country_name	VARCHAR	Calculé	Nom du pays

4.5. TABLE DE DIMENSION : DIM_BRAND

Cette table référence les marques ou fabricants rattachés à chaque produit.

Nom de Colonne	Type	Source (food.parquet)	Description métier
brand_sk (PK)	INTEGER	Clé artificielle	Clé primaire unique de la dimension marque.
brand_tag	VARCHAR	brands_tags	Tag officiel de la marque associé
brand_name	VARCHAR	brands	Nom commercial de la marque ou du fabricant.

TABLE DE DIMENSION : DIM_DATE

Elle permet d'analyser les tendances et l'évolution des produits sur le marché au fil du temps.

Nom de Colonne	Type	Source (food.parquet)	Description métier
date_sk (PK)	INTEGER	Clé intelligente numérique	Clé de tri au format fixe AAAAMMDD
date_value	DATE	created_t	Date de création issue de la conversion du timestamp
year	INTEGER	Extrait de date_value	Année civile à 4 chiffres pour les filtres annuels
month	INTEGER	Extrait de date_value	Numéro du mois de l'année (compris entre 1 et 12).
month_name	VARCHAR	Calculé	Nom du mois écrit en toutes lettres

year_month	VARCHAR	Calculé	Code combiné continu (format 'AAAA-MM') pour les courbes de tendance
quarter	INTEGER	Extrait de date_value	Numéro du trimestre de l'année (compris entre 1 et 4)
Is_weekend	BOOLEAN	Extrait de date_value	Valeur qui détermine si c'est un week-end ou non

5. DATA QUALITY

5.1. RAPPORT DATA QUALITY INITIAL

Analyse de la qualité des données sources. Anomalies identifiées et quantifiées. Stratégie de traitement retenu et justifiée.

1. TAUX DE COMPLÉTUDE

Champs	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
Environmental_score	4 521 600	914 687	3 606 913	20.23
nova_group	4 521 600	1 125 764	3 395 836	24.9
nutriscore_score	4 521 600	1 364 575	3 157 025	30.18
quantity	4 521 600	1 429 215	3 092 385	31.61
nutriscore_grade	4 521 600	1 464 068	3 057 532	32.38
fiber_g	4 521 600	1 734 149	2 787 451	38.35
categories_tags	4 521 600	1 841 000	2 680 600	40.72
brands	4 521 600	2 836 153	1 685 447	62.72
salt_g	4 521 600	2 840 693	1 680 907	62.82
saturated_fat_g	4 521 600	3 017 386	1 504 214	66.73
sugars_g	4 521 600	3 074 581	1 447 019	68.0
fat_g	4 521 600	3 247 955	1 273 645	71.83
proteins_g	4 521 600	3 252 629	1 268 971	71.94
energy_kcal	4 521 600	3 275 081	1 246 519	72.43
product_name	4 521 600	4 243 559	278 041	93.85
countries_tags	4 521 600	4 498 782	22 818	99.5
code	4 521 600	4 521 599	1	100.0
created_t	4 521 600	4 521 599	1	100.0

last_modified_t	4 521 600	4 521 599	1	100.0
-----------------	-----------	-----------	---	-------

2. ANOMALIES DÉTECTÉES

Anomalie identifiée	Volume (requête Trino)	Exemple réel trouvé	Impact sur l'analyse	Traitement retenu en silver/
Code EAN manquants ou vides	1	[]	Produit non retrouvable par scan mobile.	Garder MAX(last_modified_t) : ROW_NUMBER() OVER (PARTITION BY code ORDER BY last_modified_t DESC), garder rn=1
Produits sans nom	277996	Code 7340011495437 (Nom absent / liste vide)	Fiche mobile peu lisible.	Supprimer la ligne
Marques absentes	1685447	['code' : '0000130008136', brands : NULL, ...]	Analyses par marque incomplètes.	Rechercher avec le nom du produit la marque sinon laisser unknown
Catégories 2734422 pays absents	2680600 / 22818	Code 8033520537208 (Catégories à [])	Analyses par catégorie ou pays incomplètes.	Regarder la langue du productc_name Sinon remplacer par Unknown
Nutriscore aberrant / incohérent	2824 aberrants+ 107652 incohérents	31 998 produits avec un Grade b mais un Score de 0	Risque d'affichage de fausses recommandations nutritionnelles sur l'application mobile.	Mettre NULL : CASE WHEN nutriscore_score < -15 OR nutriscore_score > 40 THEN NULL END
Energy_kcal aberrante > 900	19 763	« Milka choco cow » avec une valeur de energy_kcal a 1015384.825	Fausse totalement les moyennes de calories et les graphiques de répartition	Filtrer : WHERE energy_kcal <= 900 dans stg_nutrition.sql Justification : erreur physiquement impossible (limite = 884 kcal pour l'huile pure).

Anomalie identifiée	Volume (requête Trino)	Exemple réel trouvé	Impact sur l'analyse	Traitement retenu en silver/
				Volume faible → on supprime ces lignes.
Protéines > 100g (impossible)	1 746	« Beef Bouillon Cubes » (Knorr) avec 52 857.15 g de sel pour 100g		Mettre NULL : CASE WHEN proteins_g > 100 THEN NULL ELSE proteins_g END
Sel > 100g (erreur d'unité certaine)	2 447	« Beef Bouillon Cubes » (Knorr) avec 52 857.15 g de sel pour 100g		Mettre NULL : CASE WHEN salt_g > 100 THEN NULL ELSE salt_g END
Sel entre 65g et 100g (élevé, plausible)	5 995	« Beef Bouillon Cubes » (Knorr) avec 52 857.15 g de sel pour 100g		Flagguer : is_sel_eleve = TRUE si salt_g > 65 — conserver la valeur
NULL > 50% (salt, nova, nutriscore...)	Voir complétude	« Beef Bouillon Cubes » (Knorr) avec 52 857.15 g de sel pour 100g		Conserver NULL. Ne jamais imputer. Documenter l'exclusion dans les analyses.
Autres Nutriments erronées / aberrantes / manquantes	6 233 Graisses > 100g 6 474 Sucres > 100g	« Beef Bouillon Cubes » (Knorr) avec 52 857.15 g de sel pour 100g	Les analyses nutritionnelles deviennent incomplètes ou fausses.	Si peu de val manquant, imputer par la médiane, sinon laisser vide/ supprimer si nutriments trop peu rempli/erroné

*L'absence de donnée ne prend pas en compte les champs existants ayants une valeur « unknown »

Voici les résultats de certaines requêtes, notamment sur les calculs statistiques classiques sur chaque colonne : `SUMMARIZE SELECT * FROM v_food_base;`

[illegible]

Voici une analyse plus poussée pour voir des valeurs anormales dans les nutriments :

```
SELECT COUNT(*) AS total_produits,
-- Energie SUM(CASE WHEN energy_kcal > 900 THEN 1 ELSE 0 END) AS
energie_aberrante,
SUM(CASE WHEN energy_kcal BETWEEN 850 AND 900 THEN 1 ELSE 0 END) AS
energie_a_verifier,
-- Proteines SUM(CASE WHEN proteins_g > 100 THEN 1 ELSE 0 END) AS
proteines_impossibles,
SUM(CASE WHEN proteins_g > 85 THEN 1 ELSE 0 END) AS proteines_tres_elevees,
-- Graisses SUM(CASE WHEN fat_g > 100 THEN 1 ELSE 0 END) AS graisses_impossibles,
-- Sucres SUM(CASE WHEN sugars_g > 100 THEN 1 ELSE 0 END) AS sucres_impossibles,
-- Sel SUM(CASE WHEN salt_g > 100 THEN 1 ELSE 0 END) AS sel_impossible,
SUM(CASE WHEN salt_g > 65 THEN 1 ELSE 0 END) AS sel_tres_eleve, -- Nutri-Score
SUM(CASE WHEN nutriscore_score < -15 OR nutriscore_score > 40 THEN 1 ELSE 0 END)
AS nutriscore_hors_plage,
-- NOVA SUM(CASE WHEN nova_group NOT IN (1,2,3,4) AND nova_group IS NOT NULL THEN
1 ELSE 0 END) AS nova_invalide
FROM v_food base;
```

	123 total_produits	123 energie_aberrante	123 energie_a_verifier	123 proteines_impossibles	123 proteines_tres_elevees	123 graisses_impossibles
1	4 521 600	19 763	18 931	1 746	8 612	6 233

Régénérer Save Cancel Exporter les résultats ... 200 1 1 row(s) fetched - 20s (0,001s fetch), on 2026-06-1

123 sucres_impossibles	123 sel_impossible	123 sel_tres_eleve	123 nutriscore_hors_plage	123 nova_invalide
6 474	2 447	5 995	2 824	0

Voici maintenant des incohérences de composition des produits :

```
SELECT product_name, brands, proteins_g, fat_g, salt_g,
ROUND(proteins_g + fat_g + salt_g, 1) AS somme_sans_glucides
FROM v_food_base
WHERE (proteins_g + fat_g + salt_g) > 110
AND proteins_g IS NOT NULL
AND fat_g IS NOT NULL
AND salt_g IS NOT NULL
ORDER BY somme_sans_glucides DESC
LIMIT 20;
```

	product_name	AZ brands	123 proteins_g	123 fat_g	123 salt_g	123 somme_sans_glucides
	AZ lang	AZ text				
7			[NULL]	1 820 151	5 028 218	8 428 102
8	en	القاسم	ناب نؤكيلك	8 578 293	90 937	1 000
9	main	wheels	Balaji	75	21,5	355 942,5
10			[NULL]	0	111 111	1 000
11	main	zeszyt	ekipa	100 000	2	0
12	main	Sal Alta Pureza	Refisal	0	0	98 020
13			[NULL]	0	0	97 500
14	main	Monster Energy Zero Ult	Monster Energy	0	0	92 500
15	main	h	[NULL]	8	2	86 999
16	main	Riced cauliflower	Season's Choice	2,3529413	0	58 823,53
17	main	Beef Bouillon Cubes	Knorr	28,57	14,29	52 857,15
18	main	Perfect Amino	BodyHealth	43 800	0	0
19	main	Viande de grison	[NULL]	40 939	3,3	3,4
20	main	מלח הארץ	מלח הארץ	0	0	39 000

Nous avons également recensé des erreurs entre le Nutriscore_score et le nutriscore grade. Une des deux données est donc fausse, nous devons trouver laquelle pour corriger l'erreur et ne pas biaiser l'interprétation, même si le nombre de produits concernés est assez faible et peu représentatif :

```
SELECT lower(nutriscore_grade) AS nutriscore_grade, nutriscore_score, COUNT(*)
AS nb_produits FROM v_food_base
WHERE nutriscore_grade IS NOT NULL AND nutriscore_score IS NOT NULL
AND (
-- Grade A : le score doit être <= 0 (Anomalie si > 0)
(lower(nutriscore_grade) = 'a' AND nutriscore_score > 0)
-- Grade B : le score doit être entre 1 et 2 (Anomalie si hors plage)
OR (lower(nutriscore_grade) = 'b' AND (nutriscore_score < 1 OR
nutriscore_score > 2))
-- Grade C : le score doit être entre 3 et 10 (Anomalie si hors plage)
OR (lower(nutriscore_grade) = 'c' AND (nutriscore_score < 3 OR
nutriscore_score > 10))
-- Grade D : le score doit être entre 11 et 18 (Anomalie si hors plage)
OR (lower(nutriscore_grade) = 'd' AND (nutriscore_score < 11 OR
nutriscore_score > 18))
-- Grade E : le score doit être >= 19 (Anomalie si < 19)
OR (lower(nutriscore_grade) = 'e' AND nutriscore_score < 19))
GROUP BY lower(nutriscore_grade), nutriscore_score
ORDER BY nb_produits DESC
LIMIT 20;
```

	AZ nutriscore_grade	123 nutriscore_score	123 nb_produits
1	b	0	31 998
2	b	-1	8 598
3	d	8	6 786
4	d	7	6 253
5	e	11	6 166
6	e	12	6 157
7	b	-2	5 197
8	e	13	5 168
9	e	10	4 724
10	d	9	4 513
11	e	14	3 898
12	b	-3	3 382
13	b	-5	3 060
14	b	-4	2 729
15	e	15	2 256
16	e	16	1 468
17	e	17	908
18	e	18	826
19	b	-6	304
20	a	4	83

Régénérer Save Cancel

Enfin, nous avons trouvé des doublons de code EAN, il y en a peu (5) donc ça ne sera pas un problème majeur, nous devons juste supprimer les lignes en double.

```
SELECT code, COUNT(*) AS nb_doublons, MIN(product_name::VARCHAR) AS nom_exemple,
MIN(last_modified_t) AS premiere_version,
MAX(last_modified_t) AS derniere_version
FROM v_food_base
GROUP BY code
HAVING COUNT(*) > 1
ORDER BY nb_doublons DESC
LIMIT 100;
```

	AZ code	123 nb_doublons	AZ nom_exemple	123 premiere_version	123 derniere_version
56	5010482907839	2	[[{'lang': 'main', 'text': 'BBQ Chicken Pasta'}, {'lang': 'en', 'text': 'BBQ Chicken Pasta'}]]	1 661 105 049	1 774 641 568
57	8020053501622	2	[[{'lang': 'main', 'text': 'Yogurt mirtillo'}, {'lang': 'it', 'text': 'Yogurt mirtillo'}]]	1 659 122 974	1 774 915 256
58	7340011485437	2	[]	1 610 393 709	1 720 542 434
59	0059527601559	2	[[{'lang': 'main', 'text': 'Coconut, Cranberry, Brazil Nut Snack Bar Variety Pack (Canada)'}, {'lang': 'en', 'text': 'Coconut, Cranberry, Brazil Nut Snack Bar Variety Pack (Canada)'}, {'lang': 'fr', 'text': 'Brazil Nut'}]]	1 732 542 251	1 734 130 218
60	8033520537208	2	[]	1 626 173 624	1 774 905 361

6. PIPELINE ELT (APACHE HOP)

Le pipeline créé sur Apache Hop joue un rôle dans la phase d'extraction (E) et de chargement (L) de données de notre architecture ELT.

Le flux duplique 100 % des données en parallèle pour alimenter d'un côté le stockage analytique (Data Lake) et de l'autre le stockage applicatif (MongoDB).

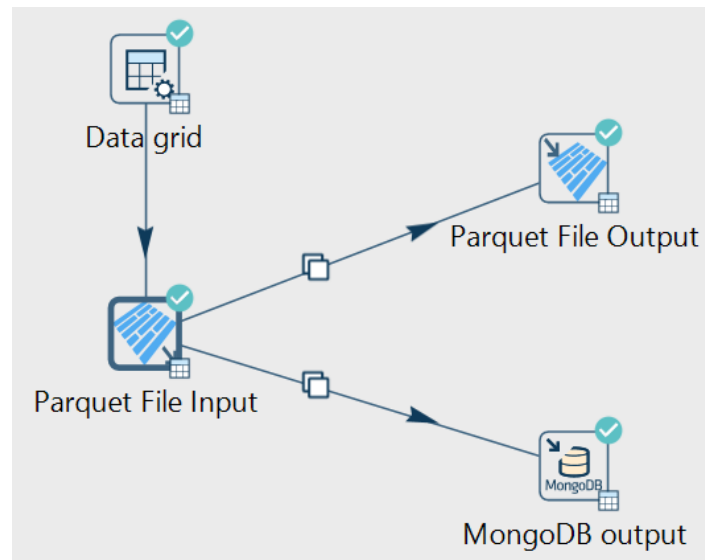


Figure 8 : Pipeline ELT sur Apache Hop

6.1. ÉTAPE PRÉALABLE : PRÉPARER LE FICHIER SOURCE POUR HOP

Avant d'en arriver là, il a été nécessaire de trouver des solutions aux limites du moteur local de Hop face aux structures imbriquées de gros fichiers (bug d'overflow Java), qu'on retrouve dans le fichier food.parquet.

En effet ce fichier contient des colonnes de types complexes (ARRAY, STRUCT imbriqué) que Hop ne sait pas lire nativement. Avant de construire le pipeline, il faut produire un fichier Parquet "plat" avec uniquement des types scalaires simples. C'est le rôle de DuckDB dans cette étape. On prépare et aplatit le fichier en amont via une requête où l'on cible les colonnes qui ont besoin d'être mis à plat.

```
COPY (
  SELECT
    -- 1. Attributs pour DIM_PRODUCT
    code,
    product_name::VARCHAR          AS product_name,
    nutriscore_grade,
    nova_group,
    environmental_score_grade,
    product_quantity,

    -- 2. Attributs pour DIM_CATEGORY
    categories,
    categories_tags::VARCHAR        AS categories_tags,

    -- 3. Attributs pour DIM_COUNTRY
```

```

countries_tags::VARCHAR          AS countries_tags,

-- 4. Attributs pour DIM_BRAND
brands,
brands_tags::VARCHAR            AS brands_tags,

-- 5. Attributs pour DIM_DATE
created_t,
last_modified_t,

-- 6. Indicateurs pour FACT_NUTRITION (Aplatissage de la structure
nutriments)
nutriscore_score::INTEGER        AS nutriscore_score,
CAST(list_filter(nutriments, x -> x.name = 'energy-kcal')[1]['100g'] AS
DOUBLE) AS energy_kcal_100g,
CAST(list_filter(nutriments, x -> x.name = 'fat')[1]['100g']          AS
DOUBLE) AS fat_100g,
CAST(list_filter(nutriments, x -> x.name = 'saturated-fat')[1]['100g'] AS
DOUBLE) AS saturated_fat_100g,
CAST(list_filter(nutriments, x -> x.name = 'sugars')[1]['100g']       AS
DOUBLE) AS sugar_100g,
CAST(list_filter(nutriments, x -> x.name = 'salt')[1]['100g']         AS
DOUBLE) AS salt_100g,
CAST(list_filter(nutriments, x -> x.name = 'proteins')[1]['100g']     AS
DOUBLE) AS proteins_100g,
CAST(list_filter(nutriments, x -> x.name = 'fiber')[1]['100g']        AS
DOUBLE) AS fiber_100g
FROM read_parquet('C:/data/nutriScan/sources/food.parquet')
WHERE code IS NOT NULL
) TO 'C:/data/nutriScan/sources/food_clean.parquet' (FORMAT PARQUET);

```

6.2. DESCRIPTION DES COMPOSANTS

1. Data Grid (Générateur de flux)

- **Rôle** : Injecter le chemin d'accès physique du fichier à traiter dans le flux.
- **Configuration** : Il contient une unique ligne envoyant la chaîne de texte C:\data\nutriScan\sources\food_clean.parquet. Ce composant est indispensable car l'injecteur Parquet de Hop est orienté flux et exige de recevoir le nom du fichier depuis une étape amont pour déclencher sa lecture.

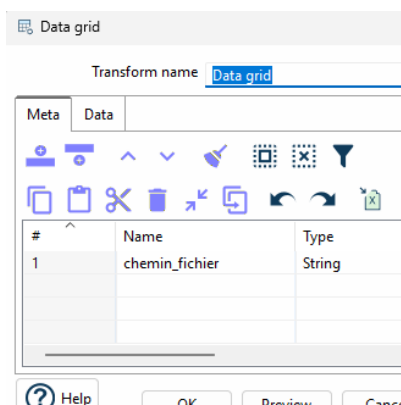


Figure 9 : Composant Data grid - Meta

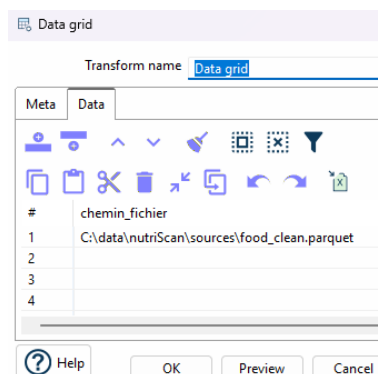


Figure 10 : Composant Data grid - Data

2. Parquet File Input (Extract)

- **Rôle** : Ouvrir le fichier binaire préparé et charger les données en mémoire.
- **Configuration** : Il récupère le chemin dynamique envoyé par le Data Grid. Grâce au bouton *Get Fields*, il mappe automatiquement les colonnes du fichier .parquet nettoyées (sans les types ARRAY et STRUCT imbriqué).

Parquet File Input

Transform name: Parquet File Input

Filename field: chemin_fichier

Metadata filename: C:\data\nutriScan\sources\food_clean.parquet

Fields

#	Source field	Target field	Type	Format
1	code	code	String	
2	product_name	product_name	String	
3	nutriscore_grade	nutriscore_grade	String	
4	nova_group	nova_group	Integer	
5	environmental_score_...	environmental_score...	String	
6	product_quantity	product_quantity	String	
7	categories	categories	String	
8	categories_tags	categories_tags	String	
9	countries_tags	countries_tags	String	
10	brands	brands	String	
11	brands_tags	brands_tags	String	
12	created_t	created_t	Integer	
13	last_modified_t	last_modified_t	Integer	
14	nutriscore_score	nutriscore_score	Integer	
15	energy_kcal_100g	energy_kcal_100g	Number	
16	fat_100g	fat_100g	Number	
17	saturated_fat_100g	saturated_fat_100g	Number	
18	sugar_100g	sugar_100g	Number	
19	salt_100g	salt_100g	Number	
20	proteins_100g	proteins_100g	Number	
21	fiber_100g	fiber_100g	Number	

Help OK Get Fields Cancel

Figure 11 : Composant Parquet File input

3. Parquet File Output (Load)

- **Rôle** : Écrire les données dans la couche cible de notre Data Lake local.
- **Configuration** : Paramétré sur le chemin C:\data\nutriScan\raw\food. Les options *Split into parts* (découpage) et *Include transform copy number* ont été désactivées afin d'obtenir un fichier unique et complet.

Parquet File Output

Transform name: Parquet File Output

Filename

Base file name: C:\data\nutriScan\raw\food

Extension:

Include date: ☐

Include time: ☐

Include date-time format: ☐

Date time format: yyyyMMdd-HHmss

Include transform copy number: ☐

Split into parts and include number: ☐

Split size: 1000000

Create parent folders: ☒

Compression codec: UNCOMPRESSED

Version: Parquet 2.0

Row group size: 268435456

Data page size: 8192

Dictionary page size: 1048576

Fields

#	Source field	Target field
1	code	code
2	product_name	product_name
3	nutriscore_grade	nutriscore_grade
4	nova_group	nova_group
5	environmental_score_grade	environmental_score_grade
6	product_quantity	product_quantity
7	categories	categories
8	categories_tags	categories_tags
9	countries_tags	countries_tags
10	brands	brands
11	brands_tags	brands_tags
12	created_t	created_t
13	last_modified_t	last_modified_t
14	nutriscore_score	nutriscore_score
15	energy_kcal_100g	energy_kcal_100g
16	fat_100g	fat_100g
17	saturated_fat_100g	saturated_fat_100g
18	sugar_100g	sugar_100g
19	salt_100g	salt_100g
20	proteins_100g	proteins_100g
21	fiber_100g	fiber_100g

Help OK Get Fields Cancel

Figure 12 : Composant Parquet File output

4. MongoDB Output (Load - Cas d'usage Mobile)

- **Rôle** : Alimenter la base de données orientée documents pour le Profil 3 (Utilisateur de l'application mobile).
- **Configuration** : Connecté sans identifiants sur notre instance locale (localhost:27017) vers la base *nutriscan* et la collection *products_raw*. Le tableau des colonnes récupérés par le composant input peut être filtré manuellement sur Hop pour ne conserver que les 10 colonnes descriptives et nutritionnelles indispensables. En soit, ne garder que les données utiles au cas d'usage mobile. La case *Truncate collection* est activée afin de vider la base avant chaque exécution pour éviter des doublons et un surplus de données.

MongoDB output

Transform name: MongoDB output

Output options | MongoDB document fields | Create/drop indexes

MongoDB Connection: MongoDB_local

Collection: products_raw

Batch insert size: 100

Truncate collection: ☒

Update: ☐

Upsert: ☐

Multi-update: ☐

Modifier update: ☐

Number of retries for write operations: 5

Delay, in seconds, between retry attempts: 10

Figure 13 : MongoDB output – output options

MongoDB output

Transform name: MongoDB output

Output options | MongoDB document fields | Create/drop indexes

#	Name	Mongo document path	Use field name	NULL values	JSON	Match field for update	Modifier operation	Modifier policy
1	chemin_fichier		Y	Ignore	N	N	N/A	Insert&Update
2	code		Y	Ignore	N	N	N/A	Insert&Update
3	product_name		Y	Ignore	N	N	N/A	Insert&Update
4	nutriscore_grade		Y	Ignore	N	N	N/A	Insert&Update
5	nova_group		Y	Ignore	N	N	N/A	Insert&Update
6	environmental_score_grade		Y	Ignore	N	N	N/A	Insert&Update
7	product_quantity		Y	Ignore	N	N	N/A	Insert&Update
8	categories		Y	Ignore	N	N	N/A	Insert&Update
9	categories_tags		Y	Ignore	N	N	N/A	Insert&Update
10	countries_tags		Y	Ignore	N	N	N/A	Insert&Update
11	brands		Y	Ignore	N	N	N/A	Insert&Update
12	brands_tags		Y	Ignore	N	N	N/A	Insert&Update
13	created_t		Y	Ignore	N	N	N/A	Insert&Update
14	last_modified_t		Y	Ignore	N	N	N/A	Insert&Update
15	nutriscore_score		Y	Ignore	N	N	N/A	Insert&Update
16	energy_kcal_100g		Y	Ignore	N	N	N/A	Insert&Update
17	fat_100g		Y	Ignore	N	N	N/A	Insert&Update
18	saturated_fat_100g		Y	Ignore	N	N	N/A	Insert&Update
19	sugar_100g		Y	Ignore	N	N	N/A	Insert&Update
20	salt_100g		Y	Ignore	N	N	N/A	Insert&Update
21	proteins_100g		Y	Ignore	N	N	N/A	Insert&Update
22	fiber_100g		Y	Ignore	N	N	N/A	Insert&Update

Get fields | Preview document structure

Figure 14 : MongoDB output – Mongo document fields

6.3. CHARGEMENT DES DONNÉES BRUTES SUR RAW

Face à la problématique de type de données du fichier food.parquet dans le pipeline sur Apache Hop, il a fallu trouver un autre moyen pour charger les données brutes dans notre répertoire « raw », sans qu'il y soit la moindre transformation effectuée sur la source. J'ai donc effectué cette opération directement sur DuckDB qui lui n'a pas de problème à gérer ce type de données.

```
COPY (
  SELECT * FROM 'C:/data/nutriScan/sources/food.parquet'
) TO 'C:/data/nutriScan/raw/food.parquet' (FORMAT PARQUET);
```

Updated Rows	4521600
Execute time	16m 35s
Start time	Thu Jun 18 14:03:31 CEST 2026
Finish time	Thu Jun 18 14:20:07 CEST 2026
Query	COPY (SELECT * FROM 'C:/data/nutriScan/sources/food.parquet') TO 'C:/data/nutriScan/raw/food.parquet' (FORMAT PARQUET)

Figure 15 : Résultat Chargement données vers raw

6.4. VERSIONNEMENT ET LOGS D'EXÉCUTION

Afin de répondre à la question concernant la relance du pipeline demain matin sur un nouveau fichier Open Food Facts, et ce qui se passe avec le fichier food_clean.parquet d'aujourd'hui et les logs pour des jours spécifiques, c'est là qu'intervient le partitionnement par date et la table pipeline_logs.

Fichier horodate en silver

La commande COPY ... TO de DuckDB exige une chaîne de caractères fixe et littérale, incapable d'évaluer la concaténation (||) ou strftime(). La syntaxe peut fonctionner sur des moteurs distribués, mais pas sur DuckDB.

```
COPY (
  SELECT
    code,
    product_name::VARCHAR AS product_name,
    nutriscore_grade,
    nova_group,
    environmental_score_grade,
    product_quantity,
    categories,
    categories_tags::VARCHAR AS categories_tags,
    countries_tags::VARCHAR AS countries_tags,
    brands,
    brands_tags::VARCHAR AS brands_tags,
    created_t,
    last_modified_t,
    nutriscore_score::INTEGER AS nutriscore_score,
    CAST(list_filter(nutriments, x -> x.name = 'energy-kcal')[1]['100g'] AS
DOUBLE) AS energy_kcal_100g,
    CAST(list_filter(nutriments, x -> x.name = 'fat')[1]['100g'] AS
DOUBLE) AS fat_100g,
    CAST(list_filter(nutriments, x -> x.name = 'saturated-fat')[1]['100g'] AS
DOUBLE) AS saturated_fat_100g,
    CAST(list_filter(nutriments, x -> x.name = 'sugars')[1]['100g'] AS
DOUBLE) AS sugar_100g,
    CAST(list_filter(nutriments, x -> x.name = 'salt')[1]['100g'] AS
DOUBLE) AS salt_100g,
    CAST(list_filter(nutriments, x -> x.name = 'proteins')[1]['100g'] AS
DOUBLE) AS proteins_100g,
    CAST(list_filter(nutriments, x -> x.name = 'fiber')[1]['100g'] AS
DOUBLE) AS fiber_100g
```

```
FROM read_parquet('C:/data/nutriScan/sources/food.parquet')
WHERE code IS NOT NULL
) TO 'C:/data/nutriScan/silver/food_clean_20260619.parquet' (FORMAT PARQUET);
```

J'ai donc ajouté la date du jour en brut.



Figure 16 : Propriété fichier food_clean_date.parquet

```
SELECT COUNT(*)
FROM read_parquet('C:/data/nutriScan/silver/food_clean_20260622.parquet');
```

Tableau	123 count_star()
1	4 521 600

Figure 17 : Résultat vérif si food_clean_date lisible

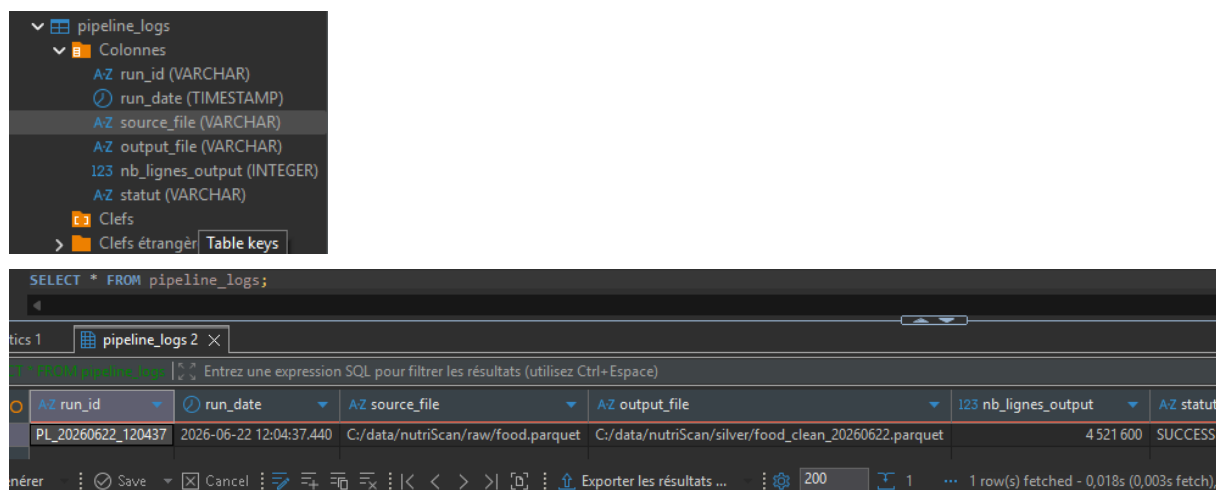
Table pipeline_logs dans DuckDB

Pour obtenir les logs sur l'état et la volumétrie de nos pipelines à une date précise, il a fallu ajouter, créer et alimenté une table de logs dédiée dans DuckDB.

```
-- Création de la structure de métadonnées
CREATE TABLE IF NOT EXISTS pipeline_logs (
    run_id VARCHAR,
    run_date TIMESTAMP,
    source_file VARCHAR,
    output_file VARCHAR,
    nb_lignes_output INTEGER,
    statut VARCHAR
);

-- Injection automatique après exécution réussie du run
INSERT INTO pipeline_logs
VALUES (
    'PL_' || strftime(now(), '%Y%m%d_%H%M%S'),
    now(),
    'C:/data/nutriScan/raw/food.parquet',
```

```
'C:/data/nutriScan/silver/food_clean_' || strftime(now(), '%Y%m%d') ||
'.parquet',
(SELECT COUNT(*) FROM read_parquet('C:/data/nutriScan/silver/food_clean_' ||
strftime(now(), '%Y%m%d') || '.parquet')),
'SUCCESS'
);
```



The screenshot shows a database interface with a table named 'pipeline_logs'. The table schema is displayed on the left, and the query result is shown in a table below.

Table Schema:

- Columns:
 - run_id (VARCHAR)
 - run_date (TIMESTAMP)
 - source_file (VARCHAR)
 - output_file (VARCHAR)
 - nb_lignes_output (INTEGER)
 - statut (VARCHAR)
- Keys:
 - Table keys

Query Result:

run_id	run_date	source_file	output_file	nb_lignes_output	statut
PL_20260622_120437	2026-06-22 12:04:37.440	C:/data/nutriScan/raw/food.parquet	C:/data/nutriScan/silver/food_clean_20260622.parquet	4 521 600	SUCCESS

Figure 18 : Requête test sur pipeline_logs

7. BASE DE DONNÉES NOSQL MONGODB

7.1. JUSTIFICATION DU CHOIX

Tout d'abord, l'utilisation de MongoDB dans notre architecture répond directement aux besoins du Profil utilisateur 3, (l'utilisateur de l'application mobile NutriScan). Il souhaite pouvoir scanner un code-barres pour obtenir la fiche produit complète associée.

Bien que DuckDB soit un bon moteur analytique pour traiter de gros volumes de données, il n'est pas adapté à ce type de requêtes unitaires destinées à une API mobile performante. Pour ce besoin de recherche ciblé, MongoDB est idéal, car en indexant notre collection sur le code EAN, il garantit un temps de réponse rapide et fluide pour l'application mobile. Cela justifie son besoin, car on ne veut pas que les durées des requêtes prennent du temps pour chaque scan.

De plus, un des autres avantages de cette base orientée document est la flexibilité de son schéma face à la structure complexe et variable des données d'Open Food Facts. Par exemple avec les données dans « nutriments », le format JSON de MongoDB permet d'organiser sous forme de sous-objets imbriqués ou même d'ajouter des champs optionnels uniquement sur certains produits, sans casser la structure globale de la base. On peut donc concevoir une collection dédiée à l'application mobile en conservant seulement les champs prioritaires pour l'affichage de la fiche produit.

7.2. MONGODB CHARGÉ

L'exécution de la pipeline a été un succès. J'ai donc pu vérifier sur MongoDB que les données étaient bien là. J'ai donc réalisé ces tests via mongosh.

Comme on le voit sur la capture d'écran, l'exécution de la commande « `db.products_raw.countDocuments()` » retourne un total de 4 521 600 documents, ce qui confirme le succès du transfert de la totalité des données du fichier source. De plus, la requête « `db.products_raw.findOne()` » renvoie bien un document JSON propre, avec tout les champs choisis dans le composant sur Apache Hop. La base NoSQL est donc opérationnelle.

Comme on le voit, une première sélection a été réalisé avec une vingtaine de colonnes clés. Bien que normalement il devrait y figurer l'ensemble des 100 colonnes dans notre collection brute, mais cela demandé d'écrire manuellement chaque champ dans la requête initial. Un autre filtrage sera nécessaire afin de ne garder seulement les 10 qui sont nécessaire.

```
>_MONGOSH
> use nutriscan
< switched to db nutriscan
> db.products_raw.countDocuments()
< 4521600
> db.products_raw.findOne()
exit
< [Function (anonymous)] { help: [Function (anonymous)] Help }
> db.products_raw.findOne()
< {
  _id: ObjectId('6a31541f1217b24a7d341ae7'),
  chemin_fichier: 'C:\\data\\nutriScan\\sources\\food_clean.parquet',
  code: '0000101209159',
  product_name: "[{'lang': main, 'text': Véritable pâte à tartiner no",
  nutriscore_grade: 'e',
  environmental_score_grade: 'd',
  product_quantity: '350',
  categories: 'Petit-déjeuners,Produits à tartiner,Produits à tartiner',
  categories_tags: "['en:breakfasts', 'en:spreads', 'en:sweet-spreads",
  countries_tags: "['en:france']",
  brands: 'Bovetti',
  brands_tags: "['xx:bovetti']",
  created_t: Long('1519297017'),
  last_modified_t: Long('1774461913'),
  nutriscore_score: Long('25'),
  energy_kcal_100g: 617,
  fat_100g: 48,
  saturated_fat_100g: 10,
  sugar_100g: 32,
  salt_100g: 0.009999999776482582,
  proteins_100g: 8
}
```

Figure 19 : Résultat requête findone() - MongoDB

7.3. STRUCTURE JSON DOCUMENTÉE POUR LA COLLECTION MOBILE

Champs à GARDER pour l'application mobile :

Champs	Justification métier
code	Clé de recherche principale lors du scan du code-barres EAN en magasin.
product_name	Nom de marque et libellé commercial affichés sur la fiche produit de l'application.
brands	Marque principale permettant l'identification visuelle par l'utilisateur.
nutriscore_grade	Lettre du Nutri-Score (A à E) pour restituer l'indicateur visuel principal.
nutriscore_score	Score nutritionnel numérique fin, indispensable pour trier et classer les alternatives saines.
nova_group	Indicateur du degré de transformation industrielle requis par les professionnels de santé.
categories_tags	Tableau de tags de catégorisation permettant de requêter des alternatives de même famille.
countries_tags	Permet de filtrer et valider la disponibilité commerciale du produit selon le pays.
energy_kcal_100g, fat_100g, saturated_fat_100g, sugar_100g, salt_100g, proteins_100g, fiber_100g	Les 7 repères nutritionnels obligatoires affichés dans le tableau de composition de la fiche mobile.

Champs à EXCLURE :

Champs	Justification métier
chemin_fichier	Champ interne au pipeline, sans valeur pour l'utilisateur mobile
created_t / last_modified_t	Timestamps techniques non affichés dans l'app mobile

Champs	Justification métier
brands_tags, categories	Redondant avec brands et categories_tags
environmental_score_grade	Optionnel pour la v1 mobile — peut être ajouté plus tard

Structure JSON du document dans la collection :

```
{
  "code": "0000130008136",
  "product_name": "Véritable pâte à tartiner noisettes chocolat noir",
  "brands": "Bovetti",
  "nutriscore_grade": "e",
  "nutriscore_score": 25,
  "nova_group": 4,
  "categories_tags": ["en:breakfasts", "en:spreads", "en:sweet-spreads"],
  "countries_tags": ["en:france"],
  "energy_kcal_100g": 617,
  "fat_100g": 48,
  "saturated_fat_100g": 10,
  "sugar_100g": 32,
  "salt_100g": 0.009,
  "proteins_100g": 8,
  "fiber_100g": null
}
```

7.4. REQUÊTES MONGODB

Requête 1 : Compter et regrouper (\$match + \$group)

Cette requête permet de comptabiliser et de classer les données selon leur nutriscore_grade, soit par lettre allant de « a » à « e », avec le groupement également d'une catégorie pour lequel le score est inconnu et une autre où ce n'est pas applicable. Cette requête est une réponse aux attentes du Profil 1 pour obtenir une vision de la qualité nutritionnelle globale des produits.


```

>_MONGOSH
< switched to db nutriscan
> db.products_raw.aggregate([
  { $match: { nutriscore_grade: { $ne: null } } },
  { $group: {
    _id: '$nutriscore_grade',
    nb: { $sum: 1 }
  }},
  { $sort: { nb: -1 } }
])
< {
  _id: 'unknown',
  nb: 3013827
}
{
  _id: 'e',
  nb: 383846
}
{
  _id: 'd',
  nb: 341935
}
{
  _id: 'c',
  nb: 283335
}
{
  _id: 'a',
  nb: 198404
}
{
  _id: 'b',
  nb: 157055
}
{
  _id: 'not-applicable',
  nb: 99493
}

```

Figure 20 : Requête 1 - MongoDB

Requête 2 : Schéma flexible (\$exists)

Dans cette requête, on analyse la somme de complétude réel du score environnemental. Cela permet d'évaluer si oui ou non ce champ à assez de données pour l'exploiter par la suite.

```

> db.products_raw.countDocuments({
  environmental_score_grade: { $exists: true, $ne: null }
})
< 3685646
nutriscan>

```

Figure 21 : Requête 2 - MongoDB

Requête 3 : Filtrage de l'API de recherche (\$regex + \$in)

Cette requête recherche les produits d'une marque spécifique filtrés sur les meilleures notes nutritionnelles, dans notre cas on recherche tous les produit Danone avec Nutri-Score A ou B. Cette logique permettrait de proposer instantanément des alternatives plus saines à l'utilisateur qui cherche à faire attention.

```
>_MONGOSH
> db.products_raw.find({
  brands: { $regex: 'Danone', $options: 'i' },
  nutriscore_grade: { $in: ['a', 'b'] }
}).limit(5).pretty()
< {
  _id: ObjectId('6a31541f1217b24a7d34942e'),
  chemin_fichier: 'C:\\data\\nutriScan\\sources\\food_clean.parquet',
  code: '0034360269500',
  product_name: "[{'lang': 'main', 'text': 'Mousse au fromage blanc sur lit de fruits'}, {'lang': 'en', 'text': 'Mousse au fromage blanc sur lit de fruits'}]",
  nutriscore_grade: 'a',
  environmental_score_grade: 'c',
  categories: 'Produits laitiers, Produits fermentés, Desserts, Produits laitiers fermentés',
  categories_tags: "['en:dairies', 'en:fermented-foods', 'en:fermented-milk-products']",
  countries_tags: "['en:france']",
  brands: 'Danone',
  brands_tags: "['xx:danone']",
  created_t: Long('1534772695'),
  last_modified_t: Long('1774464152'),
  nutriscore_score: Long('1'),
  energy_kcal_100g: 55,
  fat_100g: 0.4000000059604645,
  saturated_fat_100g: 0.30000001192092896,
  sugar_100g: 5.400000095367432,
  salt_100g: 0.11999999731779099,
  proteins_100g: 6.300000190734863
}
{
  _id: ObjectId('6a3154201217b24a7d349f4f'),
  chemin_fichier: 'C:\\data\\nutriScan\\sources\\food_clean.parquet',
  code: '0036632013170',
  product_name: "[{'lang': 'main', 'text': 'ORIGINAL vanilla'}, {'lang': 'en', 'text': 'ORIGINAL vanilla'}]",
  nutriscore_grade: 'a',
  nova_group: Long('4'),
  environmental_score_grade: 'b',
  categories: 'Dairies, Fermented foods, Desserts, Fermented milk products, Dairy desserts',
  categories_tags: "['en:dairies', 'en:fermented-foods', 'en:fermented-milk-products']",
  countries_tags: "['en:united-states']",
  brands: 'DANONE'.
```

Figure 22 : Requête 3 – MongoDB

Requête 4 : Performances et Indexation (Cas d'usage EAN)

Le cas d'usage central est la recherche directe par code-barres, voici une simulation et mesure de cette requête.

```
db.products_raw.find({ code: '3017620422003' }).explain('executionStats')
```

```

executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 9872,
  totalKeysExamined: 0,
  totalDocsExamined: 4521600,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: {
      code: {
        '$eq': '3017620422003'
      }
    }
  }
}

```

Figure 23 : Requête avant index – MongoDB

Afin de simuler les conditions réelles de l'application mobile NutriScan, j'ai audité les performances de cette requête avant et après la mise en place d'un index indexé.

```

executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 24,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 1,
    executionTimeMillisEstimate: 11,
  }
}

```

Figure 24 : Requête après index – MongoDB

On peut voir que le temps d'exécution est passé de 9872 à 24 ms, en passant de 4 millions de docs examinés, à seulement 1. Avec des millions d'utilisateurs qui scannent des produits simultanément en magasin, sans index cela ferait certainement crasher le serveur MongoDB. Faire un scan complet (COLLSCAN) à chaque requête saturerait les ressources.

Comparaison avec DuckDB :

J'ai réalisé cette requête sur DuckDB afin de comparer le temps d'exécution entre les deux outils.

```

SELECT code, product_name, nutriscore_grade, nutriscore_score, nutriments
FROM v_food
WHERE code = '3017620422003';

```

Le temps d'exécution a été de 3,07 secondes.

Ce benchmark prouve que DuckDB est excellent pour les analyses mais un scan complet pour un seul produit prend plusieurs secondes. En effet un parquet est incapable de servir une application mobile en magasin : un client ne peut pas attendre plusieurs secondes pour chaque scan. MongoDB avec un index répond en 24 ms, soit un traitement bien plus rapide.

Requêtes d'agrégation

On trie par ordre croissant (1) car dans l'échelle du Nutri-Score, plus le score est bas (proche de -15), meilleure est la qualité nutritionnelle. Le filtre { \$ne: null } sans lui, les produits n'ayant pas de score calculé seraient considérés comme valant 0 par MongoDB, ce qui fausserait complètement la moyenne réelle des catégories.

```
db.products_raw.aggregate([
  {
    $match: {
      countries_tags: /en:france/i,
      // Sécurité : on n'accepte QUE les scores physiquement possibles (-15
à 40)
      nutriscore_score: { $exists: true, $ne: null, $gte: -15, $lte: 40
    }
  },
  {
    $group: { _id: '$categories_tags', score_moyen: { $avg:
'$nutriscore_score' }, nb_produits: { $sum: 1 }
  }
  },
  { $sort: { score_moyen: 1 } },
  { $limit: 3 }
])
```

```
{
  "_id": "['en:plant-based-foods-and-beverages', 'en:plant-based-foods', 'en:legumes-and-their-products', 'en:cereals-and-potatoes', 'en:legumes', 'en:seeds', 'en:cereals-and-their-products', 'en:ile
score_moyen: -15,
nb_produits: 1
}
{
  "_id": "['en:plant-based-foods-and-beverages', 'en:plant-based-foods', 'en:fruits-and-vegetables-based-foods', 'en:legumes-and-their-products', 'en:legumes', 'en:seeds', 'en:vegetables-based-foods
score_moyen: -15,
nb_produits: 1
}
{
  "_id": "['en:plant-based-foods-and-beverages', 'en:plant-based-foods', 'en:legumes-and-their-products', 'en:legumes', 'en:seeds', 'en:legume-seeds', 'en:pulses', 'en:lentils', 'en:blonde-lentils',
score_moyen: -15,
nb_produits: 1
}
```

Figure 25 : Requête d'agrégation – MongoDB

7.5. COLLECTION SCAN_LOGS ET REQUÊTE ALTERNATIVES

Pour enregistrer l'activité de scan des utilisateurs et sauvegarder immédiatement les alternatives qui leur ont été suggérées, on ajoute un script d'agrégation et de journalisation unitaire dans mongosh

```

nutriscan> // 1. Initialisation de la collection de traçabilité
db.createCollection('scan_logs');

// 2. Sélection d'un produit mal noté (Grade D ou E) ayant des catégories
var produit = db.products_raw.findOne({
  nutriscore_grade: { $in: ['d', 'e'] },
  categories_tags: { $ne: null }
});

// 3. Recherche de 3 alternatives optimales (A ou B) dans la même catégorie
var alternatives = db.products_raw.find({
  categories_tags: produit.categories_tags,
  nutriscore_grade: { $in: ['a', 'b'] },
  code: { $ne: produit.code }
}, { product_name: 1, nutriscore_grade: 1, nutriscore_score: 1, _id: 0 })
.sort({ nutriscore_score: 1 }).limit(3).toArray();

// 4. Génération et insertion du log applicatif complet
db.scan_logs.insertOne({
  scan_date: new Date(),
  code_ean: produit.code,
  product_name: produit.product_name,
  nutriscore_grade: produit.nutriscore_grade,
  nb_alternatives: alternatives.length,
  alternatives: alternatives
});

// 5. Affichage du résultat pour validation
db.scan_logs.find().pretty();

```

Figure 26 : Script collection scan_logs - MongoDB

```

< {
  _id: ObjectId('6a3a4f58e53ab08a66bb2cd4'),
  scan_date: 2026-06-23T09:18:16.529Z,
  code_ean: '0000101209159',
  product_name: "[{'lang': 'main', 'text': 'Véritable pâte à tartiner noisettes chocolat noir'}, {'lang': 'fr', 'text': 'Véritable pâte à tartiner noisettes chocolat noir'}]",
  nutriscore_grade: 'e',
  nb_alternatives: 3,
  alternatives: [
    {
      product_name: "[{'lang': 'main', 'text': 'Nocciutella'}, {'lang': 'it', 'text': 'Nocciutella'}]",
      nutriscore_grade: 'a',
      nutriscore_score: Long('1-3')
    },
    {
      product_name: "[{'lang': 'main', 'text': 'Pate a tartiner aux noisettes'}, {'lang': 'en', 'text': 'Chocolate Brownie'}, {'lang': 'fr', 'text': 'Pate a tartiner aux noisettes'}]",
      nutriscore_grade: 'a',
      nutriscore_score: Long('1-3')
    },
    {
      product_name: "[{'lang': 'main', 'text': 'Pâte à tartiner'}, {'lang': 'fr', 'text': 'Pâte à tartiner'}]",
      nutriscore_grade: 'a',
      nutriscore_score: Long('1-3')
    }
  ]
}

```

Figure 27 : Résultat du script scan_logs

Réponses aux questions de réflexion architecturale NoSQL :

1. Quel est l'intérêt d'imbriquer directement les alternatives trouvées dans le document de log plutôt que de faire une jointure au moment de la lecture ?

Cela évite de recalculer une jointure lourde plus tard et assure des performances de lecture optimales pour afficher l'historique des scans de l'utilisateur.

2. Comment la startup NutriScan peut-elle exploiter la collection scan_logs pour identifier les produits de mauvaise qualité les plus fréquemment scannés afin d'alerter les organismes de santé ?

En exécutant un pipeline d'agrégation \$group sur le champ code avec un filtre \$match ciblant les grades d et e. En triant par le compte des occurrences (\$sum: 1), NutriScan peut générer un classement en temps réel des produits les plus consommés mais les moins sains du marché.

8. CONCLUSION

En suivant la méthodologie Kimball, nous avons conçu un schéma en étoile basé sur un grain strict (un produit x un pays), répondant précisément aux besoins de nos différents utilisateurs.

J'ai débuté l'implémentation de l'architecture médaillon par la couche Bronze/Raw. L'utilisation combinée de DuckDB pour le prétraitement et d'un Data Grid sur Hop s'est révélée être utile pour le bon fonctionnement du pipeline. Cela a permis de dupliquer et charger l'intégralité des 4 521 600 lignes de la base Open Food Facts vers nos deux cibles :

Répertoire Raw : un fichier Parquet complet et stocké localement dans notre Data Lake, préservant l'historique sans altération.

MongoDB : une collection MongoDB avec une première optimisation pour garantir un choix de donnée déjà filtré.

Enfin, le rapport de Data Quality mené met en lumière plus de 100 000 incohérences massives sur le Nutri-Score et des milliers d'aberrations physiques sur les macronutriments. Ces résultats confirment la pertinence de notre approche ELT. Dans le prochain livrable, l'objectif sera d'exploiter la puissance de dbt Core au sein de la couche Silver pour appliquer des règles de gestion et de nettoyage.

9. INSTRUMENT DE RECHERCHE

9.1. FICHE DÉMOGRAPHIQUE — À REMPLIR S1 LUNDI MATIN

Code participant (attribué par l'encadrante)	
Âge	25

Formation actuelle	4 ^e année diplôme d'ingénieur informatique
Niveau en programmation Python (0=aucun, 1=débutant, 2=intermédiaire, 3=avancé)	1
Expérience préalable avec SQL (0=jamais, 1=cours, 2=projet, 3=professionnel)	3
Outils data déjà utilisés (cocher) : Excel SQL Python/pandas Power BI Tableau Autre :	Excel, SQL, Python/pandas, Power BI
As-tu déjà entendu parler de Big Data avant ce stage ? (oui/non)	Oui
As-tu déjà utilisé Docker ou des conteneurs ? (oui/non)	Oui
Motivation principale pour ce stage (1-2 phrases libres)	Montée en compétence en big data et participer à l'amélioration d'un bloc école effectué dans le passé

9.2. AUTO-POSITIONNEMENT – À REMPLIR DEUX FOIS

Remplir la colonne Initial le lundi de S1. Remplir la colonne Final le mercredi de S5.

Échelle : 0 = jamais entendu parler • 1 = entendu parler, jamais pratiqué • 2 = pratiqué en projet guidé • 3 = capable de le faire seul sur un nouveau cas

Domaine	Détail	Initial S1	Final S5
1. Données & sources	Formats (CSV, JSON, Parquet), structurées/semi-structurées	2	0 1 2 3
2. Stockage & bases	Schéma relationnel, SQL avancé, SQL/NoSQL, data warehouse vs data lake	2	0 1 2 3
3. Pipelines & traitement	Ingestion, ELT/ETL, transformation, nettoyage	2	0 1 2 3
4. Outils & écosystème	Python data, dbt, Git, notebooks	2	0 1 2 3

Domaine	Détail	Initial S1	Final S5
5. Infrastructure & scalabilité	Calcul distribué, Docker, cloud, orchestration	2	0 1 2 3
6. Qualité & gouvernance	Détection d'anomalies, valeurs manquantes, RGPD, data catalogue	2	0 1 2 3
7. Visualisation & restitution	Choix de visualisation, interprétation, communication	1	0 1 2 3

9.3. QUESTIONNAIRE CLT – S1 + S2

Échelle : 1 = Pas du tout • 2 = Plutôt non • 3 = Neutre • 4 = Plutôt oui • 5 = Tout à fait

CHARGE INTRINSÈQUE — Complexité du contenu	
Q1 — Les concepts techniques de cette semaine m'ont semblé complexes à comprendre.	2
Q2 — J'ai eu du mal à relier les nouvelles notions à ce que je savais déjà.	3
CHARGE EXTRINSÈQUE — Complexité des outils	
Q3 — Les outils ont ajouté de la complexité sans rapport avec l'apprentissage.	1
Q4 — J'ai perdu du temps sur des problèmes d'outils plutôt que sur la compréhension.	1
Q5 — J'ai pu travailler de manière autonome sans aide externe constante.	4
CHARGE GERMANE — Apprentissage effectif	
Q6 — À la fin de cette semaine, j'ai l'impression d'avoir vraiment compris ce que j'ai fait.	4
Q7 — Je serais capable d'expliquer à quelqu'un ce que j'ai construit cette semaine.	4
Q8 — Le travail de cette semaine m'a donné envie d'approfondir.	4

Question ouverte S1 — Ce qui t'a le plus freiné / le plus aidé cette semaine :

Ce qui m'a le plus freiné : Les notions de structure imbriquée dans le fichier food.parquet, manipuler et aplatir des données complexes. Installation de Java 17 (à cause de ma version antérieur)

Ce qui m'a le plus aidé : Le workshop m'as permis de comprendre des nouvelles notions dans un point de vue pratique. Les explications de Sonia, notamment sur la méthodologie Kimball et l'intérêt de fixer le grain de la table de faits à « une ligne par produit x pays » via un UNNEST

CHARGE INTRINSÈQUE — Complexité du contenu

Q1 — Les concepts techniques de cette semaine m'ont semblé complexes à comprendre.

3

Q2 — J'ai eu du mal à relier les nouvelles notions à ce que je savais déjà.

4

CHARGE EXTRINSÈQUE — Complexité des outils

Q3 — Les outils ont ajouté de la complexité sans rapport avec l'apprentissage.

2

Q4 — J'ai perdu du temps sur des problèmes d'outils plutôt que sur la compréhension.

4

Q5 — J'ai pu travailler de manière autonome sans aide externe constante.

3

CHARGE GERMANE — Apprentissage effectif

Q6 — À la fin de cette semaine, j'ai l'impression d'avoir vraiment compris ce que j'ai fait.

4

Q7 — Je serais capable d'expliquer à quelqu'un ce que j'ai construit cette semaine.

4

Q8 — Le travail de cette semaine m'a donné envie d'approfondir.

2

Question ouverte S2 — Ce qui t'a le plus freiné / le plus aidé cette semaine :

Ce qui m'a le plus freiné : J'ai mis un peu de temps à prendre en main l'interface graphique d'Apache Hop, notamment pour créer le projet ou comprendre certains champs comme les champs *Filename field* et *Metadata filename*. Mais le plus gros frein, ça a été l'erreur d'overflow Java (*Negative initial size*). Le moteur local de Hop a totalement saturé en essayant de lire les métadonnées et les structures imbriquées (STRUCT, ARRAY) du fichier du fichier source

food.parquet. Ça m'a bloqué une bonne journée, car j'ai testé différentes approches dont notamment une réduction de la taille du fichier. Un autre souci auquel j'ai fait face, c'est concernant la récupération des données d'entrées, le composant « Parquet Input » lisait 0 lignes, il avait besoin d'un intermédiaire, j'ai donc ajouté le composant « Data Grid ». Il y a eu aussi des erreurs d'authentification, le fait qu'il n'y avait pas d'utilisateur pour se connecter à la base MongoDB (le workshop indiquait un admin).

Ce qui m'a le plus aidé : L'idée de prétraitement des données avec DuckDB. Au lieu de chercher à réduire la taille du fichier ou forcer la main sur Apache Hop, cela a simplifié la démarche. En forçant la conversion des colonnes complexes en chaînes simples (VARCHAR). Ça m'a aidé aussi à comprendre les différences d'utilisation ces 2 outils.

9.4. GRILLE D'AUTO-OBSERVATION – S1 + S2

Cette grille capture tes comportements concrets de la semaine — pas tes ressentis (c'est le journal de bord), mais les faits observables. Sois précis et factuel.

GRILLE D'AUTO-OBSERVATION — Semaine S1	
Rubrique	Ta réponse
Temps total de travail cette semaine (heures estimées)	5h / jour
Temps passé sur des problèmes d'installation/configuration vs sur le fond technique (ex: 2h config / 5h technique)	30 min config / 5h30 technique (moyenne par jour)
Nombre de fois où tu as demandé de l'aide (encadrante, internet, IA) — donne une estimation	Vingtaine de fois
Outils ou commandes qui t'ont posé le plus de difficulté cette semaine (sois précis)	Installation de Java 17, Comprendre le format parquet et l'ajustement des requêtes associé
Moment où tu t'es senti le plus autonome cette semaine — décris brièvement	Un peu sur tout mais surtout sur la partie installation

As-tu utilisé une IA générative ? Si oui, pour quoi et combien de fois ?

Oui, pour approfondir ma compréhension des nouvelles notions (avec des exemples) – Entre 10-15

GRILLE D'AUTO-OBSERVATION — Semaine S2

Rubrique	Ta réponse
Temps total de travail cette semaine (heures estimées)	5h / jour
Temps passé sur des problèmes d'installation/configuration vs sur le fond technique (ex: 2h config / 5h technique)	1 journée pb config / reste du temps fond technique
Nombre de fois où tu as demandé de l'aide (encadrante, internet, IA) — donne une estimation	Une trentaine de fois (10 encadrante, 20 IA)
Outils ou commandes qui t'ont posé le plus de difficulté cette semaine (sois précis)	Apache Hop avec les données de type STUCT, ARRAY
Moment où tu t'es senti le plus autonome cette semaine — décris brièvement	Rédaction du Livrable 1. Les attendus étaient bien énoncés.
As-tu utilisé une IA générative ? Si oui, pour quoi et combien de fois ?	Oui, pour trouver des réponses aux erreurs, améliorer/faire les requêtes, reformulation. (20 fois)

9.5. JOURNAL DE BORD — S1 + S2

5 rubriques — 10 minutes maximum. Sois concret et honnête — pas de mise en scène.

JOURNAL DE BORD — Semaine S1

Rubrique	Ta réponse
Observation marquante	Décris une situation concrète de la semaine : quel outil, quel problème, quelle réussite ?
Interprétation technique	Pourquoi cette situation s'est-elle produite ? Qu'as-tu compris ?
Ressenti personnel	Comment tu t'es senti cette semaine — frustré, confiant, motivé ?
Hypothèse émergente	Quel conseil donnerais-tu à un futur étudiant qui commence cette boucle ?
Question ouverte	Qu'est-ce que tu ne comprends pas encore ?

Tes réponses :

Observation :

Lors de la conception du schéma en étoile, j'ai eu du mal à déterminer comment gérer les champs multi-valeurs (pays, catégorie), donc j'ai d'abord proposé une table de pont pour gérer ça sans multiplier les lignes. Mais on m'a corrigé en m'expliquant qu'il fallait plutôt utiliser UNNEST pour adopter un grain strict d'une ligne par produit x pays, pour gérer ces champs sans devoir complexifier le schéma avec des tables de pont.

Interprétation :

Je me suis basé sur les connaissances que j'avais, en essayant de trouver une solution dans ma « zone de confort », et je n'avais pas une connaissance assez aguerrie de ce que peut faire DuckDB. J'en ai compris, qu'il est conçu pour agréger des millions de lignes instantanément grâce à des filtres sur clés numériques. J'ai mieux compris la différence entre une BDD classique et un Datawarehouse qui ne perd pas en performance à cause de plus de ligne.

Ressenti :

Je me suis senti confiant et motivé. Je renforce mes connaissances passées, en accompagnant de nouvelles notions. J'ai le sentiment que les notions sont abordables à la compréhension si je fais l'effort de « mettre ma tête dedans »

Hypothèse :

Bien prendre le temps de comprendre les notions complexes, comme le format « .parquet », les particularités de DuckDB et son moteur OLAP, analyse des besoins pour modélisation. Toute les choses pour lesquelles, on n'a généralement pas envie de prendre le temps, ces du temps qui vaut le coup d'être pris.

Question :

La façon dont nous allons utiliser UNNEST et la notion de grain d'une ligne par produit x pays

JOURNAL DE BORD — Semaine S2

Rubrique	Ta réponse
Observation marquante	Décris une situation concrète de la semaine : quel outil, quel problème, quelle réussite ?
Interprétation technique	Pourquoi cette situation s'est-elle produite ? Qu'as-tu compris ?
Ressenti personnel	Comment tu t'es senti cette semaine — frustré, confiant, motivé ?
Hypothèse émergente	Quel conseil donnerais-tu à un futur étudiant qui commence cette boucle ?
Question ouverte	Qu'est-ce que tu ne comprends pas encore ?

Tes réponses :

Observation :

Les erreurs sur Apache Hop d'overflow Java (Negative initial size) en essayant de lire les types complexes (STRUCT, ARRAY) du fichier food.parquet brut, qui m'a fait penser et tester si c'était à cause de la taille trop importante du fichier. Ainsi que la découverte des anomalies métiers massives (plus de 100 000 Nutri-Scores incohérents avec des moyennes impossibles, des millions de lignes à « NULL » pour certaines colonnes...etc). J'ai réussi à contourner le problème en aplatisant le fichier avec DuckDB avant de l'envoyer dans Hop.

Interprétation :

Cette situation s'est produite parce que les outils graphiques basés sur Java comme Apache Hop saturent très vite en mémoire. Donc face à des données imbriqués et massifs, ça ne fonctionne pas. Mais cela m'a permis de comprendre l'intérêt et le rôle de chaque outil ainsi que l'utilité d'un prétraitement avant d'envoyer les données tête baissée dans le flux. On laisse DuckDB gérer l'aplatissement et le typage en VARCHAR, pour que Hop puisse effectuer son travail de déplacement fluide des flux vers le Data Lake et MongoDB. Concernant les anomalies, j'ai compris qu'on ne peut jamais tirer des conclusions sans les traiter et nettoyer les données.

Ressenti :

De la frustration face aux problèmes que j'ai eue sur Hop qui avec le recul ne demandez pas autant de temps à régler, mais cela m'a permis de rechercher des solutions et essayer de comprendre le problème. Mais lorsque les données étaient accessibles et requêtables depuis le raw et MongoDB, plus de motivation pour la suite.

Hypothèse :

Ne perds pas ton temps à essayer de forcer un outil d'ingestion à faire de la transformation lourde ou à utiliser des fichiers complexes mal formatés. L'importance d'un prétraitement, peut faire gagner du temps par la suite.

Question :

Je ne comprends pas encore certaines requêtes de Mongosh et de DuckDB complexe.